

Autonomous Inductive Charging System for Battery-operated Electric Drones

Majid Khonji

Masdar Institute of Science and Technology
Abu Dhabi, UAE
mkhonji@masdar.ac.ae

Chien-Ming Tseng

Masdar Institute of Science and Technology
Abu Dhabi, UAE
ctseng@masdar.ac.ae

Mohammed Alshehhi

Mohammed Bin Rashid Space Centre
Dubai, UAE
mohammed.alshehhi@mbrsc.ae

Chi-Kin Chau

Masdar Institute of Science and Technology
Abu Dhabi, UAE
ckchau@masdar.ac.ae

ABSTRACT

Electric drones have a wide range of applications due to the convenience and agility. One of the main hurdles for their wide deployment and full automation is the battery life limitation and the need for manual intervention for charging. In this work, we present a solar powered charging system capable of automatically charging battery-powered drones in remote locations. We developed a tethered robotic rover equipped with a 2D Lidar sensor to detect and localize a drone, and a robotic arm equipped with an inductive charging pad. The charging pad senses current measurements from individual inductive coils, and adjusts accordingly the robotic arm position to maximize charging rate. Such system, in principal, can cater for arbitrary drone shapes and landing positions and is less susceptible to external lighting and environmental conditions by the use of Lidar and current sensors instead of computer vision. We remark that our system can be also applied to charging other small electric vehicles.

CCS CONCEPTS

• **Computer systems organization** → **Robotic autonomy**; *Sensors and actuators*; • **Computing methodologies** → **Object detection**; • **Hardware** → **Renewable energy**; *Batteries*;

KEYWORDS

Drones, Robotics, Electric Vehicles, Inductive charging

ACM Reference format:

Majid Khonji, Mohammed Alshehhi, Chien-Ming Tseng, and Chi-Kin Chau. 2017. Autonomous Inductive Charging System for Battery-operated Electric Drones. In *Proceedings of e-Energy '17, Shatin, Hong Kong, May 16-19, 2017*, 6 pages.
DOI: <http://dx.doi.org/10.1145/3077839.3078462>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

e-Energy '17, Shatin, Hong Kong

© 2017 Copyright held by the owner/author(s). Publication rights licensed to ACM.
978-1-4503-5036-5/17/05...\$15.00

DOI: <http://dx.doi.org/10.1145/3077839.3078462>

1 INTRODUCTION

Drones (also known as unmanned aerial vehicles) are becoming a convenient means of logistics. In particular, the civilian markets for drones are forecasted to be booming globally. The notable areas of drone applications include delivery (e.g., for light-weight parcels, medical items, airmail) and remote operations (e.g., farming, environmental surveying, search and rescue operations). Remarkably, oil and gas companies and utility providers, which rely on extensive surveillance, measurements, mapping and surveying, maintenance operations for geographically diverse facilities, will be the primary customers of drones for carrying out remote missions involving minimal transportation of manpower to hazardous environments.

Despite increasingly popular uses of drones in the civilian sectors, most drones are typically controlled by human operators, using remote controllers. Piloting a drone is intrinsically difficult for human. Manual controls not only incur inefficiency and inconvenience but also are vulnerable to human errors. Often, intensive training for drone piloting are required in advance, which are usually expensive and laborious. On the other hand, improper human controls will cause drone crashes and damages. There are many repetitive routine missions that are more suitably carried out by drones, and hence, it is desirable to dispense with human controls.

Although autopilot features have been recently built into many consumer drones, the main barrier for the wide deployment of autonomous drone systems is the short battery life. Multi-rotor drones, in particular, have a short flight time, ranging from 15 up to 40 minutes only. On the other hand, charging a drone typical requires human intervention which restricts full automation.

Recently, several commercial companies (e.g., Skysense) developed drone charging systems that do not require human intervention. These systems rely on tailor-made drone hardware and software that provide the capability and physical interface of accurate landing on a charging dock. However, most consumer drones lack such capabilities, and are mainly equipped with commercial GPS and Inertial Measurement Unit (IMU) that suffer from landing inaccuracy (up to 3 meters) and orientation drifts. Although accurate landing can be achieved by solely using computer vision techniques, it requires computing resources and protocols that are not yet standardized in commercial drones. Moreover, computer vision techniques are susceptible to lighting and texture changes due to weather conditions which make them less robust with uncertainties. Accurate landing can be also achieved by Real Time

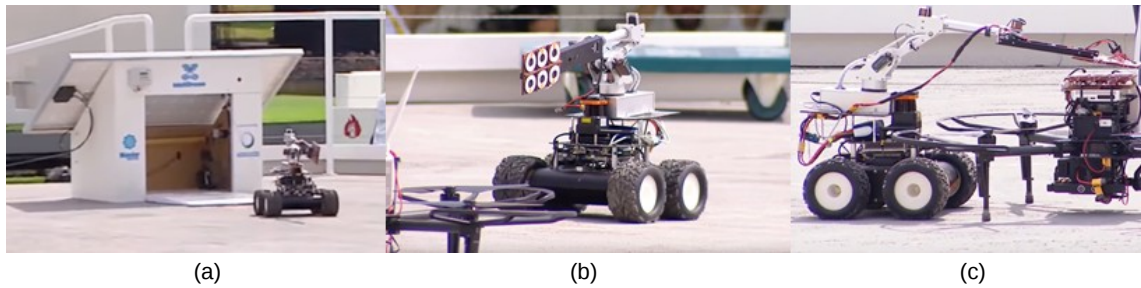


Figure 1: (a) The charging system is powered entirely by a solar panel. (b) The rover detects the drone and autonomously navigates to its location. (c) The robotic arm adjusts the inductive charging pads and starts to charge the drone.

Kinematic (RTK) satellite navigation technique for precise outdoor localization; however, it requires a ground base station and a steady communication link with the drone, which may not be suitable for the case of remote charging stations. In addition, the landing-based charging systems constrain the shape and design of a drone to match that of the charging station. We believe an alternate way is more effective, by making the charging station adjustable to different drones.

In this work, we propose an adjustable automatic charging system. We propose a system comprises of three components: (1) A solar-powered station that can be deployed in remote locations. (2) A tethered rover capable of autonomous navigation, equipped with a robotic arm equipped with a charging pad that can adapt to different drone sizes and landing positions. The inductive charging pad senses current measurements of individual inductive coils, and accordingly, used to adjust the arm positions to maximize charging rate. (3) Drones equipped with thin inductive receivers can land at arbitrary locations within a landing zone for charging (see Fig. 1). Our robotic rover uses 2D Light Detection and Ranging (Lidar) sensor to navigate and detect a drone. Lidar sensors are more robust to environmental uncertainties and external lighting conditions than computer vision sensors.

The paper is structured as follows. In Sec. 2 we present the related work, followed by a conceptual view of our system and algorithm pipeline in Sec. 3. In Secs. 4-6 we present our algorithm pipeline, namely, drone segmentation and detection, autonomous navigation, and inductive charging, respectively. The conclusion and future remarks are provided in 7.

2 RELATED WORK

Several areas of research intersect with this work, namely, Lidar point clustering and segmentation, autonomous navigation, and inductive charging.

The problem of Lidar point cloud segmentation and clustering has been widely studied in the context of autonomous vehicles. We highlight few earlier works related to our system. One popular approach is graph-based clustering using a Euclidean Minimum Spanning Tree (EMST) [15]. This class of algorithms builds a minimum spanning tree between the points, then decides which edge to break in order to obtain disjoint clusters. For instance, it was shown in [1] that by removing $K - 1$ largest edges from the spanning tree, a clustering is obtained which maximizes the minimum inter-cluster

distance between K groups. A seminal work by Felzenszwalb et.al. [4] considers a more robust approach that produces more perceptually coherent segments using two general cost metrics: *internal* and *external difference* between and within clusters. Their algorithm run in $O(n \log n)$ time which makes it attractive in practical applications. Several more recent works improve on the aforementioned algorithms (see, e.g., [14], [13]).

For autonomous navigation and path planning, we rely on Robotic Operating System (ROS) [6], Navigation Stack, in our prototype implementation, which implements several Simultaneous Localization and Mapping (SLAM) algorithms. For a more in-depth survey about SLAM, we refer the reader to [2].

Our main contribution in this work is due to the system architecture and the use of current sensor measurements to refine the inductive charging procedure. We also present a fast segmentation algorithm that is suitable for uncluttered environments, with a single compact object such as a drone. The simplicity of the algorithm makes it attractive in this context.

3 SYSTEM OVERVIEW

In this section, we illustrate the main components of our system, namely, the charging station, the robotic rover, and the drone; followed by our hardware setup, and an overview of the charging algorithm process. The system is comprised of three main components (Fig. 1):

- **Charging station:** Since the goal of such system is to charge drones in remote areas, disconnected from the grid, it is vital to provide self-generating charging station, as well as to protect the robotic rover from harsh environment conditions (i.e., rain, the wind, etc.) when idle. As shown in Fig. 1a, our initial prototype is entirely powered by a solar panel that stores energy into a 50000 watt-hour solar battery. The solar housing powers the electronics, rover, charge drones and protects the robotic rover. A landing zone with an area of at least $3 \times 3 \text{ m}^2$ is designated for drone landing in front of the solar housing.
- **Robotic rover:** To enable flexible deployment, a tethered rover is equipped with a robotic arm that mounts a wireless inductive transmitter (see Fig. 2). Using autonomous drone detection and navigation, the rover can cater for arbitrary landing position and orientation of a drone, which is the case with most commercial drones that solely rely

on GPS and commercial Inertial Measurement Unit (IMU) for landing.

- **Drones:** A drone is equipped with an inductive wireless receiver, compatible with that of the robotic rover. In order to charge a drone, the drone, or a fleet management system [12], should send a signal to the charging station to initiate the charging procedure.

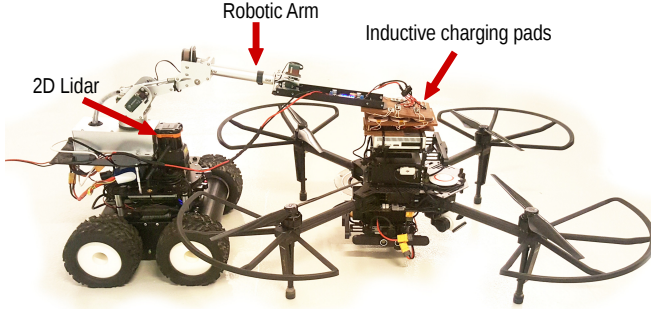


Figure 2: Robotic charger

3.1 Hardware Setup

We use Hokuyo UTM-30LX Scanning Laser Rangefinder to obtain 2D point cloud. The laser rangefinder provides 30 meters detection range which allows the rover to detect far objects such as drones or obstacles. Detection angle is 270° with 0.25° angular resolution. The rover odometry is obtained based on wheel rotation using quadrature encoders.

A retractable arm is used to conduct inductive charging for the drones¹. The inductive charging technology is based on “resonant inductive coupling”, which can transfer energy without physical contact. It has been widely used in wireless charging for smart phones. The use of inductive charging technology facilitates a fully automated fleet management system, without physically plugging an external charger. The current prototype uses six coils for inductive charging, giving a maximum charging speed of $6 \times 700 = 3500$ mA. We mounted six current sensors, on the robotic arm, to measure individual coil charging rate.

Our test drone is DJI Matrice 100 which has an on-board Linux computer running ROS (shown in Fig. 3). The drone is powered by two 5700 mA on-board Li-Poly batteries.



Figure 3: Our test drone: DJI Matrice 100 with Nvidia Manifold onboard Linux box.

¹We use a customize version of a consumer robotic arm called 7bot (www.7bot.cc)

3.2 Algorithm Process

Once a drone is landed and charging command is sent, the robotic rover proceeds as follows:

- (1) *Finding the drone:* From raw 2D lidar point cloud, we use a clustering technique to separate drone points from background points. Further details illustrated in Sec. 4.1.
- (2) *Classifying the drone and detecting its orientation:* Based on supervised learning, drone information (e.g., type, size, orientation, etc.) can be extracted from point cloud data. In this work, we consider a single drone, and its orientation is obtained using a simple rectangle model (details are illustrated in Sec. 4.2). Drone orientation is needed to instruct the rover to navigate to a position that supports a better alignment for the charging pad with the receiver on the drone. This allows more flexibility in the charging pad design (e.g., rectangular inductive charging pad instead of a circular pad) to fit more inductive coils on the drone.
- (3) *Autonomously navigating towards the drone:* the exact position of the navigation goal is decided based the allowable charging poses (see Sec. 5).
- (4) *Charging:* Position the charging pad on the drone and refine the positioning using current sensors readings (see Sec. 6).

Once a drone is fully charged, a termination command is sent to the charging station. The rover, then, returns to the charging station to its idle state.

4 DRONE SEGMENTATION AND ORIENTATION DETECTION

In this section, we show the first two stages of our algorithm pipeline, namely, we present the segmentation and drone orientation detection methods. Although segmentation and clustering techniques have been widely studied in the literature [1, 4], we present a simple algorithm that is more suitable for uncluttered environments, which is the case of a single drone. Our test drone (Fig. 3) has an ‘L’ or ‘T’ shaped 2D Lidar point signature, depending on its landing orientation (see Fig. 5bb). Therefore we consider a simple rectangle model to bound and detect the drone’s orientation. The orientation is, then, used in the last stage of our algorithm pipeline to proceed with charging.

4.1 Segmentation

Denote the set of all raw 2D Lidar points by $\mathcal{P} \subset \mathbb{R}^2$. Let δ be the maximum diameter of the drone points, that is, any pair of Lidar points pertaining to the drone has a pairwise distance of at most δ . We also denote μ to be the maximum distance between two consecutive Lidar points. μ is used to threshold subsequent Lidar points in order to separate them into different objects.

Our segmentation algorithm is presented in Alg. 1. The algorithm proceeds as follows: after the initialization step (Lines 1-3), the algorithm iterates over the set of all unvisited points $\mathbf{p} \in \mathcal{P} \setminus \mathcal{V}$, where $\mathcal{V}$ is the set of visited points. Starting from point $\mathbf{p}$, we grow a cluster C_i by expanding through the nearest neighbors recursively. F is the frontier set of our expansion search from point $\mathbf{p}$. In each step, we calculate the centroid and the radius of C_i denoted by $\mathbf{a}$ (Line 14) and m (Line 16), respectively. Finally, we

take cluster C_i if its radius m is at most $\frac{\delta}{2}$ (Lines 23-24), and hence the process is repeated until all points in $\mathcal{P}$ are visited. The set $\text{NEIGHBORS}(s)$ (Line 18) returns all points within distance μ from s . We note that μ can be set heuristically based on Lidar angular resolution, and the maximum expected distance between the drone and the rover. If multiple clusters of diameter δ are returned; for the sake of our application, a simple tie-breaking rule can be applied such as returning the cluster with maximum points.

Implementation remarks: Our algorithm performance depends on how we obtain the neighbors for any given point. One way to optimize is to create a minimum spanning tree $T = (V, E)$ in a preprocessing stage (a similar idea was used in [14]), where $V = \mathcal{P}$ and each edge $(u, v) \in E$ is associated with the euclidean distance between node u and v . Given a spanning tree (or a forest), the running time of our algorithm is $O(n)$. We use Kruskal's algorithm [7] to calculate the euclidean minimum spanning tree (EMST). The running time of Kruskal's algorithm is $O(|E| \log n)$; however, $|E|$ in a complete graph, corresponding to pairwise distances, is n^2 . To overcome this issue, we compute the Delaunay triangulation [3] of nodes V in $O(n \log n)$ time and $O(n)$ space to obtain a planner graph. Since there are $O(n)$ edges in a planner graph, Kruskal's algorithm requires only $O(n \log n)$ time. Hence, the total running time of Alg. 1 is $O(n \log n)$.

Algorithm 1 SEGMENT $[\mathcal{P}, \delta]$

```

1:  $C \leftarrow \emptyset$  ▷ Family of clusters
2:  $\mathcal{V} \leftarrow \emptyset$  ▷ Set of visited nodes
3:  $i \leftarrow 1$ 
4: while  $\mathcal{V} \neq \mathcal{P}$  do
5:    $C_i \leftarrow \emptyset$ 
6:    $p \leftarrow$  an arbitrary point in  $\mathcal{P} \setminus \mathcal{V}$ 
7:    $F \leftarrow \{p\}$ ;  $\mathcal{V} \leftarrow \mathcal{V} \cup \{p\}$ 
8:    $m \leftarrow 0$ ;  $k \leftarrow 0$ 
9:    $a \leftarrow (0, 0)$  ▷ Moving average centroid of cluster  $C_i$ 
10:  while  $F \neq \emptyset$  do
11:     $s \leftarrow$  an arbitrary point from  $F$ 
12:     $F \leftarrow F \setminus \{s\}$ 
13:     $C_i \leftarrow C_i \cup \{s\}$ 
14:     $a \leftarrow a + (s - a)/(k + 1)$ ;  $k \leftarrow k + 1$ 
15:    if  $\|a - s\| > m$  then
16:       $m \leftarrow \|a - s\|$ 
17:    end if
18:    for  $r \in \text{NEIGHBORS}(s) \setminus \mathcal{V}$  do
19:       $F \leftarrow F \cup \{r\}$ 
20:       $\mathcal{V} \leftarrow \mathcal{V} \cup \{r\}$ 
21:    end for
22:  end while
23:  if  $m \leq \delta/2$  then
24:     $C \leftarrow C \cup \{C_i\}$ ;  $i \leftarrow i + 1$ 
25:  end if
26: end while
27: return  $C$ 

```

4.2 Orientation Detection

Several methods are proposed in the literature to extract features from Lidar point cloud (see, e.g., [5]), such as the number of points, estimated area, and the standard deviation with principle horizontal directions. From the ensemble of features one can obtain the drone orientation. Another approach in [10] uses Iterative Closest Point (IPC) method to rotate and translate points to match a reference point cloud (see, e.g., [8]).

We observe that the point cloud pertaining to our test drone (see Fig. 3) has an 'L' or 'I' shape that can be modeled by a rectangle. Many commercial drones have similar point cloud signatures². Our robotic charging pad has a rectangular shape (Fig. 6) and can efficiently charge either from the front or back directions of the drone. Therefore, a bounding rectangle can be used to identify the front or back directions by looking at the large rectangle segments (see Fig. 5b).

To obtain the right rectangle, i.e., the one corresponding to the actual shape, we use the rotating calipers algorithm [9, 11]. The algorithm proceeds as follows:

- $C \leftarrow$ Obtain the convex hull of $\mathcal{P}$
- From each segment of C , define a minimum area rectangle $\mathcal{R}$ that contains all points of C . See Fig. 4 for a pictorial illustration.
- Iterate over all rectangles $\mathcal{R}$ and return the one with least area.

In the next section, we use the bounding rectangle to send a navigation goal to the rover.

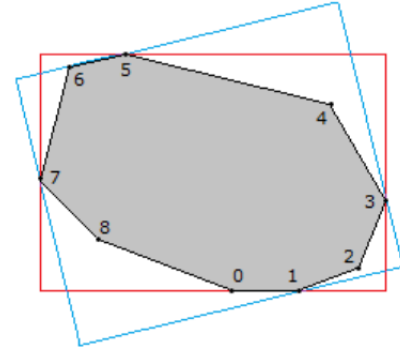


Figure 4: The gray region corresponds to the convex hull of $\mathcal{P}$, and the black segments are the boundaries. Here, we consider two segments, namely, (0, 1) and (5, 6) for which we draw a red and blue rectangles, respectively, that contain all points of the convex hull.

5 AUTONOMOUS NAVIGATION

Now we describe the third stage of our algorithm pipeline. We rely mainly on ROS Navigation Stack in our implementation. The Navigation Stack basically takes in information from the rover odometry and sensor streams and outputs velocity commands to

²We shall extend this part in future to account for arbitrarily shaped drones.

send to the robotic rover motors. It builds two obstacle cost maps, local and global, that are used to plan feasible trajectory paths for a given navigation goal coordinate.

The way we set the goal coordinate is based on the detected drone rectangle and its orientation. Given the rectangle (Sec. 4.2),

- obtain the closest of the two large segments of the rectangle, then
- find a point p such that the line connecting p to the center of the segment is orthogonal, see Fig. 5a. (Fig. 5b shows an screenshot of our ROS implementation.)

We consider a safety distance between the drone and the rover navigation goal to prevent any collision and simplify the robotic arm positioning task. (In our prototype, we set the distance to 43 cm.) Once the goal is set, the rover autonomously navigates to position p and stops heading towards the center c of the segment ready for charging.

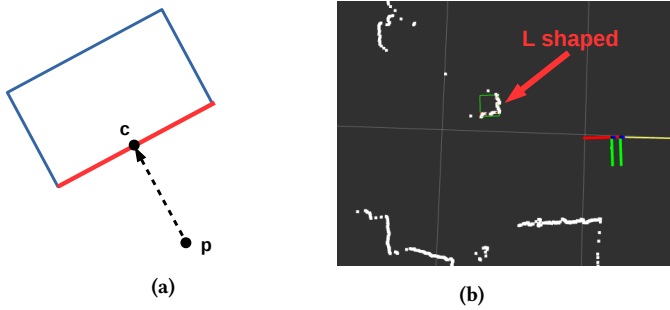


Figure 5: (a) Illustrates the location of the navigation goal p set with respect to the center of the rectangle segment c (dark dots). The length of the dash line is the safety distance between the rover and the drone. (b) ROS implementation (RVIS tool) of the segmentation and orientation algorithms: white dots are 2D Lidar point cloud. The drone is detected (green rectangle) from its 'L' shaped point cluster. The large segments of the rectangle correspond to the front/back directions.

6 INDUCTIVE CHARGER ADJUSTMENT

In this section, we illustrate our final stage of our algorithm pipeline, charging the drone. The process is twofold, (1) move the arm to a preset position, (2) adjust the arm based on current sensor readings for individual inductive coils. The inductive charging technology is based on "resonant inductive coupling", which can transfer energy without physical contact. Our inductive charging pad comprises of six coils (see Fig. 6). Each coil is capable of delivering 700 mA, a total of $6 \times 700 = 3500$ mA. Fig. 7 shows a schematic view of a typical inductive charger.

We rely on a simple deterministic approach to adjust the robotic arm: Based on the relative intensity measurements of current between the coils, the arm is commanded to turn right and left until total current is maximized.

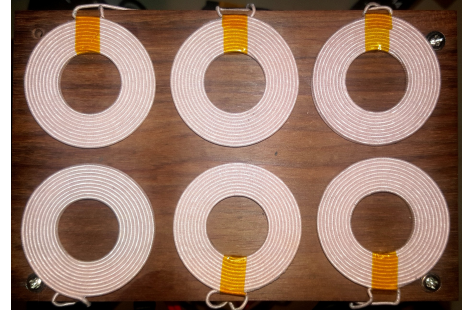


Figure 6: Our inductive charging pad.

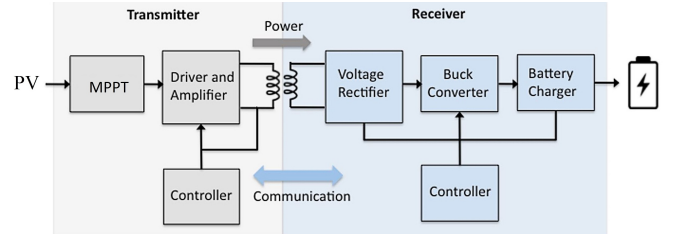


Figure 7: Inductive charger schematic.

7 CONCLUSION

Automated charging systems are useful for future application of drones to achieve full autonomy. This paper provides an autonomous charging system capable of charging battery-operated electric drones without human assistance. We presented our system architecture and algorithm pipeline, from drone segmentation and detection to inductive charging and arm adjustment. We note that our system can cater for arbitrary drone landing positions and does not depend on computer vision, hence not susceptible to external lighting and environmental conditions.

In the future, we will extend our analysis and provide further details and plots for inductive charging, arm adjustment, and loss quantification. We shall further extend our study to incorporate drones with different geometries. A flight planning system considering autonomous drone charging has been implemented in a companion paper [12].

REFERENCES

- [1] Tetsuo Asano, Binay Bhattacharya, Mark Keil, and Frances Yao. 1988. Clustering algorithms based on minimum and maximum spanning trees. In *Proceedings of the fourth annual symposium on Computational geometry*. ACM, 252–257.
- [2] Tim Bailey and Hugh Durrant-Whyte. 2006. Simultaneous localization and mapping (SLAM): Part II. *IEEE Robotics & Automation Magazine* 13, 3 (2006), 108–117.
- [3] Kevin Buchin and Wolfgang Mulzer. 2011. Delaunay triangulations in O (sort (n)) time and more. *Journal of the ACM (JACM)* 58, 2 (2011), 6.
- [4] Pedro F Felzenszwalb and Daniel P Huttenlocher. 2004. Efficient graph-based image segmentation. *International journal of computer vision* 59, 2 (2004), 167–181.
- [5] Aleksey Golovinskiy, Vladimir G Kim, and Thomas Funkhouser. 2009. Shape-based recognition of 3D point clouds in urban environments. In *Computer Vision, 2009 IEEE 12th International Conference on*. IEEE, 2154–2161.
- [6] Rodrigo Longhi Guimarães, André Schneider de Oliveira, João Alberto Fabro, Thiago Becker, and Vinicius Amilgar Brenner. 2016. *ROS Navigation: Concepts and Tutorial*. Springer International Publishing, Cham, 121–160. DOI: http://dx.doi.org/10.1007/978-3-319-26054-9_6

- [7] Joseph B Kruskal. 1956. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical society* 7, 1 (1956), 48–50.
- [8] Szymon Rusinkiewicz and Marc Levoy. 2001. Efficient variants of the ICP algorithm. In *3-D Digital Imaging and Modeling, 2001. Proceedings. Third International Conference on*. IEEE, 145–152.
- [9] Michael Ian Shamos. 1978. Computational geometry. (1978).
- [10] Michael Thuy and Fernando Puente Leon. 2009. Non-linear, shape independent object tracking based on 2d lidar data. In *Intelligent Vehicles Symposium, 2009 IEEE*. IEEE, 532–537.
- [11] Godfried T Toussaint. 1983. Solving geometric problems with the rotating calipers. In *Proc. IEEE Melecon*, Vol. 83. A10.
- [12] Chien-Ming Tseng, Chi-Kin Chau, Khaled Elbassioni, and Majid Khonji. 2017. *Flight Tour Planning with Recharging Optimization for Battery-operated Autonomous Drones*. Technical Report. Masdar Institute. <https://arxiv.org/abs/1703.10049>.
- [13] Dominic Zeng Wang, Ingmar Posner, and Paul Newman. 2012. What could move? finding cars, pedestrians and bicyclists in 3d laser data. In *Robotics and Automation (ICRA), 2012 IEEE International Conference on*. IEEE, 4038–4044.
- [14] Dominic Zeng Wang, Ingmar Posner, and Paul Newman. 2015. Model-free detection and tracking of dynamic objects with 2D lidar. *The International Journal of Robotics Research* 34, 7 (2015), 1039–1063.
- [15] Charles T Zahn. 1971. Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Transactions on computers* 100, 1 (1971), 68–86.